Práctica de Laboratorio 4

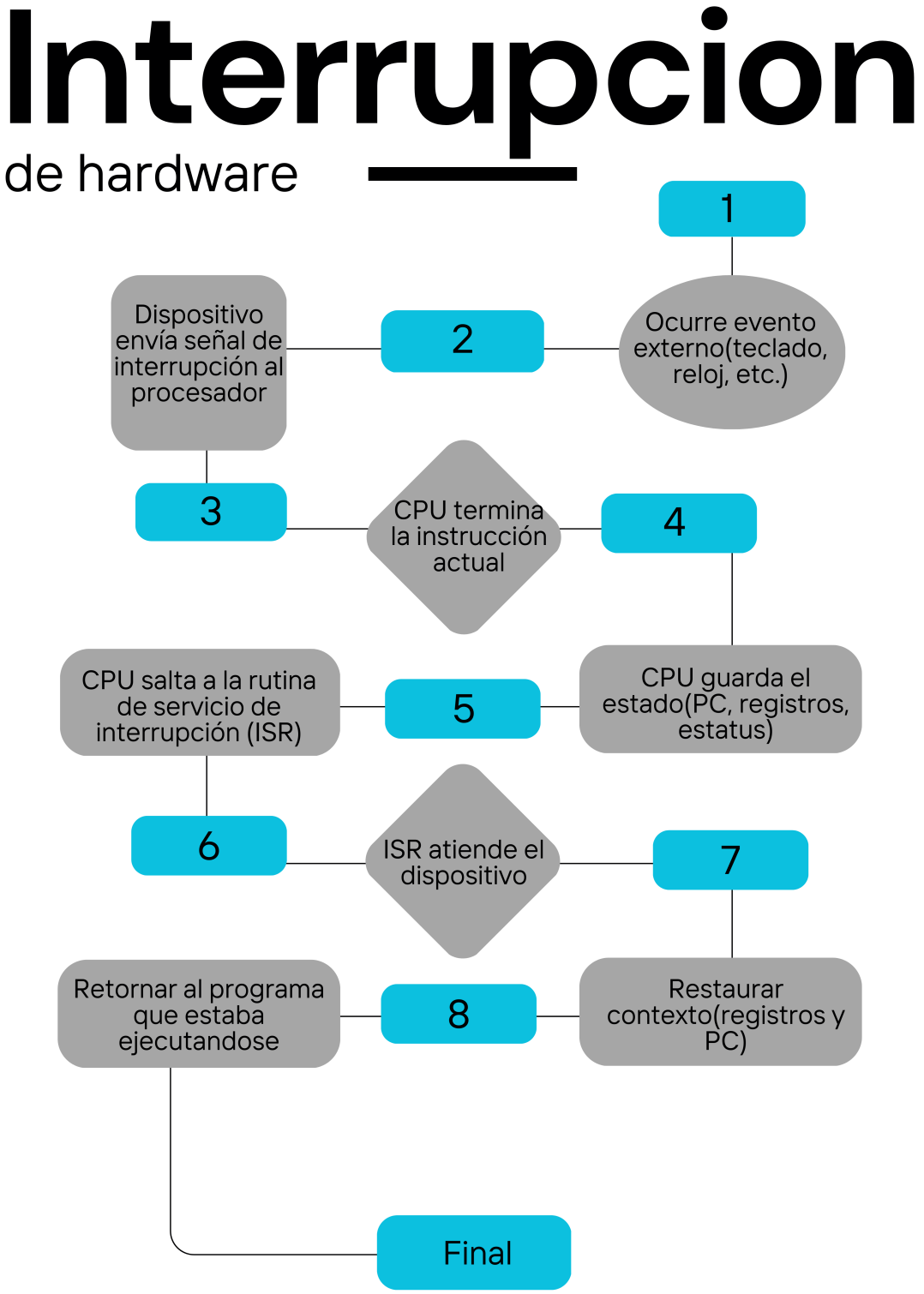
Freddy Ovalle C.I 31709650
Juan Bermmúdez C.I 31608369
Universidad de Carabobo
Facultad de Ciencias y Tecnología
Departamento de Computación

13 de Octubre de 2026

Objetivo

Familiarizar al estudiante con la sintaxis del lenguaje ensamblador MIPS32 y su ejecución en el entorno MARS, resolviendo problemas clásicos de programación, prestando especial atención al uso de interrupciones para la gestión de entrada/salida (E/S) y la utilización de rutinas de servicio para responder a eventos como entrada por teclado y control de tiempo, simulando el funcionamiento de periféricos.

1 **Explique con un diagrama cómo funciona el ciclo de una interrupción de hardware**



2 ¿Qué diferencias hay entre gestionar E/S con sondeo y hacerlo con interrupciones?

El sondeo consulta constantemente la informacion que le proporciona el dispositivo, mientras que las interrupciones permiten al procesador ocuparse de otras tareas hasta que el dispositivo notifique que necesita atención.

3 ¿Qué ventajas tiene el uso de interrupciones en términos de uso del procesador?

El uso de interrupciones permite usar de manera mas eficiente el procesador , ya que el CPU se ocupa de diferentes tareas mientras espera eventos de Entrada/Salida, por lo que no debe gastar recursos ni ocupar ciclos en consultar constantemente con los dispositivos.

4 ¿Qué registros especiales se utilizan en MIPS32 para gestionar interrupciones?

Los registros principales utilizados para gestionar interrupciones en el entorno MIPS32 son los registros especiales del coprocesador 0; siendo los mas importantes:

4.1 status

El status($12) controla el estado general de las intrucciones del procesador, este habilita o deshabilita el estatus de las interrupciones globales, e indica que tipo de interrupciones estan permitidas.

4.2 Cause

El cause($13) indica que evento provoco la interrupción o excepción, este contiene información sobre el origen del evento.

4.3 EPC

El EPC($14) guarda la dirección de la instrucción que estaba ejecutandose al momento de la interrupción, permite reanudar la ejecución del programa luego de atender la interrupción.

5 ¿Por qué es necesario guardar el contexto (registros) al entrar en una rutina de servicio?

La razón principal se debe a que la rutina de servicio al interrumpir la ejecución del programa puede cambiar los valores de los registros mientras esta interrumpido, por lo que al reanudar la ejecución se tomarían valores incorrectos o alterados, al guardar el contexto aseguramos preservar el estado de nuestros registros luego de la interrupción.

6 Momentos en que pueden generarse excepciones en un sistema MIPS32

6.1 Enumera al menos 4 situaciones en las que se pueda generar una excepción (por ejemplo: desbordamiento aritmético, fallo de dirección, etc.)

6.1.1 Desbordamiento aritmético

Cuando se maneja un valor que no se puede representar en 32 bits, ya sea muy grande o muy pequeño.

6.1.2 División por cero

Al momento de realizar una división por 0 que no se puede representar, no está definido.

6.1.3 Dirección de memoria inválida

Al intentar acceder a una dirección de memoria que no existe, no esta asignada a nada.

6.1.4 Instrucción ilegal

Al intentar utilizar una instrucción que no pertenece a la lista de instrucciones del entorno MIPS32.

6.2 Explica qué etapas del pipeline pueden provocar una excepción y por qué

En la etapa IF se obtiene la instrucción desde la memoria utilizando la dirección contenida en el contador de programa. En esta etapa puede producirse una excepción si la dirección de la instrucción es inválida o si ocurre un fallo al acceder a la memoria donde se encuentra la instrucción.

En la etapa ID se decodifica la instrucción y se leen los registros necesarios para su ejecución. En este punto puede generarse una excepción si se detecta que la instrucción es ilegal o no está definida dentro del conjunto de instrucciones del procesador.

En la etapa EX se realiza la operación aritmética o lógica correspondiente a la instrucción. Durante esta etapa puede producirse una excepción si ocurre un desbordamiento aritmético en

operaciones como suma o resta, o si se detecta alguna condición inválida relacionada con la operación que se está ejecutando.

En la etapa MEM se realiza el acceso a memoria para instrucciones que leen o escriben datos, como lw o sw. Aquí pueden producirse excepciones si la dirección de memoria es inválida o si ocurre un error de alineación en el acceso.

7 Estrategias de tratamiento de excepciones e interrupciones

7.1 Explica las diferencias entre interrupciones y excepciones, (¿son síncronas o asíncronas?)

Las interrupciones son pausas al programa destinadas a atender a dispositivos que necesitan usar el CPU,el uso de estos dispositivos forma parte de la ejecución del codigo a diferencia de las excepciones que son pausas al programa pero a causa de condiciones anormales que surgieron de imprevisto durante la ejecución, es decir no forman parte del correcto funcionamiento del programa.

las interrupciones son asincrónicas ya que pueden pasar en cualquier momento de la ejecución, no dependen de la instrucción que se esté ejecutando a diferencia de las excepciones que son sincrónicas, ya que estan relacionadas con la instrucción en ejecución.

7.2 Describe brevemente dos estrategias para tratar excepciones en un sistema MIPS32¿Cómo se redirige la ejecución hacia la rutina de servicio? ¿Cuál es la función del registro EPC (Exception Program Counter)?

7.2.1 Manejador general de excepciones

Todas las excepciones van a un mismo lugar en memoria, a una rutina única que se encarga de analizar qué pasó, por ejemplo si ocurre un desbordamiento o un acceso inválido a memoria, la rutina lee el registro Cause para saber exactamente qué pasó y luego lo atiende.

7.2.2 Rutinas específicas por tipo de excepción

Cada tipo de excepción tiene su propia rutina de atención. Cuando ocurre un error, primero se pasa por un manejador principal que luego redirige a la rutina correcta, en este caso, tomando el ejemplo de la primera estrategia si hay desbordamiento se ejecuta directamente la rutina para desbordamiento o si es el caso del acceso a memoria inválido, se ejecuta la rutina de fallo de memoria.

8 Habilitación de interrupciones en dispositivos y procesador

8.1 Explica cómo se habilitan las interrupciones en teclado, pantalla y procesador(detalla qué bits del registro Status deben modificarse)

8.1.1 Teclado

Se debe pasar a la dirección de memoria del registro de control del teclado un 2 (10).

8.1.2 Pantalla

Se debe pasar a la dirección de memoria del estado de la pantalla un 1 (01).

8.1.3 Procesador

En el procesador, para trabajar con las interrupciones de los dispositivos, es importante hacer uso de el status, si vamos a habilitar globalmente las interrupciones habilitamos el status.IE asignandole 1, sino podemos manejarlo individualmente y asignarle 1 iM0-iM7 dependiendo del que estemos utilizando, solo el que tenga uno podrá generar interrupciones. Por otra parte es importante que los registros EXL y ERL esten en 0, el primero nos dice que el procesador no esta ocupado en otra interrupción y el segundo nos indica que no haya ningun tipo de error con excepciones.

8.2 ¿Qué pasaría si habilitamos interrupciones en los dispositivos pero no en el procesador?

Si habilitas los dispositivos (por ejemplo, teclado y pantalla) pero el procesador no tiene el bit IE=1, entonces aunque el dispositivo solicite la interrupción, el procesador la ignorará porque está globalmente deshabilitada, solo cuando ambas condiciones están cumplidas, es decir, que el procesador reciba interrupciones y que el dispositivo la solicite, la interrupción será reconocida y atendida.

9 Procesamiento de interrupciones

9.1 Describe paso a paso qué ocurre cuando se produce una interrupción de reloj: Desde que el evento ocurre hasta que la rutina de servicio termina. ¿Qué registros se guardan y restauran? ¿Qué hace el hardware y qué hace el software (sistema operativo o rutina)?

Cuando se produce una interrupción de reloj en el entorno MIPS32, el hardware detecta la señal enviada por el temporizador y detiene temporalmente la ejecución del programa actual, guardando la dirección de la instrucción interrumpida en el registro EPC y cambiando a modo excepción. Luego, el procesador salta automáticamente a la rutina de servicio de interrupción correspondiente, donde el software primero guarda los registros generales y cualquier otro estado necesario para no sobrescribir los datos del programa interrumpido.

Durante la ejecución del ISR, actualiza contadores de tiempo, controla los procesos que exceden su tiempo máximo o notificar al sistema operativo para cambiar de tarea. Al finalizar, el software restaura los registros guardados y ejecuta la instrucción eret, que permite al hardware volver al flujo normal del programa interrumpido, recuperando el estado anterior y reactivando las interrupciones para que el procesador continúe ejecutando.

9.2 ¿Por qué es importante guardar el contexto (registros generales, EPC, Status) al entrar en la rutina?

Los registros generales son necesarios para conservar los valores y no alterarlos en el algoritmo principal luego de la ejecución de la rutina, gracias al EPC podemos reanudar el programa al momento donde nos llamó la interrupción pudiendo seguir con el resto del programa , y el Status nos permite verificar que no haya otra interrupción en ejecución con el bit EXL.

10 Interrupciones de reloj y control de ejecución

10.1 Explica cómo una interrupción de reloj puede usarse para: evitar bucles infinitos y finalizar programas que superan un tiempo máximo de ejecución.

Para evitar bucles infinitos o limitar el tiempo máximo de ejecución de un programa se implementa un contador que se incrementa cada vez que ocurre la interrupción y se compara con un valor límite predefinido por quien escriba el codigo, si aun no se excede el limite, el programa se ejecuta con normalidad. Si el contador supera el tiempo permitido, la rutina puede finalizar el programa de manera controlada, liberando recursos y evitando que quede bloqueado el sistema. En este proceso, el hardware se encarga de generar la señal de interrupción y de guardar el estado del procesador, mientras que el software, a través de la rutina de servicio o el sistema operativo, evalúa el tiempo transcurrido y decide si continuar la ejecución o detener el programa. De esta manera, la interrupción de reloj actúa como un moderador que controla la duración de los programas y previene la ejecución indefinida.

10.2 ¿Qué debe hacer el software si el programa finaliza antes de la interrupción de reloj?

El programa debe ignorar la interrupción de reloj y finalizar el programa de manera segura y limpia, liberando el contexto y restaurar los valores de los registros del procesador evitando que la interrupción de reloj interfiera o intente ejecutar código innecesario para un programa ya finalizado.

11 Análisis y Discusión de los Resultados

Al trabajar con estos dos algoritmos pudimos notar que no finalizan al menos que los forcemos, ya que siempre estan constantmente esperando las señales de la entrada indefinidamente , en el primer caso, leerá el teclado por 20 segundos hasta que demos por finalizado la ejecución, y en el segundo espera por la entrada de la letra "s" indefinidamente, por lo que se puede decir que la duración del algoritmo depende de cuando querramos finalizar la ejecución, su complejidad BIG-O es O(1).

En el segundo algoritmo tomamos en cuenta el valor 32 del registro$v0 para la implementación del semáforo, de esta manera no debemos comparar innesesariamente hasta llegar al tiempo de cada color del semáforo sino que simplemente detenemos la ejecución del codigo hasta que pasen los milisegundos definidos para cada uno, de esta manera se puede trabajar mas eficientemente dejando en reposo el procesador entre cada cambio de color, ya que en este lapso no se hace uso de otras operaciones.

Por otra parte, en el primer algoritmo necesitabamos del valor 30 en el registro $v0, el cual nos indica el tiempo actual de la ejecución, esto ya que no se puede colocar en reposo el procesador, en este algoritmo se necesita ir recibiendo las teclas ingresadas por la interrupción del teclado e ir comprobando si son mayusculas, es decir, hay operaciones internas en el intervalo de 20 segundos, además gracias al valor ascii de los caracteres podemos hacer uso de instrucciones de MIPS32 para comparar los caracteres ingresados por teclado gracias a su valor ascii, en el rango desde A hasta Z, simplemente comparandolos numericamente.

Finalmente, una caracteristica que pudimos notar en la elaboración del codigo, es que la interacción con el teclado se logra gracias al memmory-mapped, ya que recibimos los caracteres desde la dirección de memoria asignada al teclado, y habilitabamos el teclado mediante la dirección de memoria ffff0000, por lo que pudimos interactuar con el dispositivo haciendo uso de instrucciones simples del entorno MIPS32.